Deleted-Record Recovery from SQLite Databases:

Storage Conventions, a `forbid(unsafe)` Reference Implementation, and a Multi-Oracle Forensic Evaluation

Anonymous Submission

June 2026

Abstract

SQLite is the most widely deployed database engine in the world [1], and the deleted rows inside its files are often the evidence an investigator needs—yet the live query interface (`sqlite3`) cannot see them. This paper gives an account of where deleted data persists in a SQLite file and how to recover it defensibly. We set out the storage model and the conventions that govern residue lifetime: the comparative retirement of the rollback `-journal` in application contexts in favour of the write-ahead log (`-wal`, which retains *multiple* uncheckpointed commits), and the destructive effect of `VACUUM` and `PRAGMA secure_delete`. We describe a set of recovery mechanisms—freelist and free-block carving, exact-tile free-block reconstruction, overflow-chain reassembly, dropped-table schema recovery, a WAL commit-history overlay, and rollback-journal before-image folding—each constrained by a *live-row re-read exclusion invariant*: a row still reachable from the live b-tree is never reported as deleted.
We contribute: (1) a treatment of SQLite deletion and free-space conventions, including an application case study of 微信 (WeChat) message deletion; (2) our carver, an open, `forbid(unsafe)` Rust reference implementation; and (3) a multi-oracle evaluation—accuracy, false-positive rate, and throughput—against four independent recovery tools (`undark`, `fqlite`, `bring2lite`, and DC3's SQLite Dissect) on independent third-party corpora with published ground truth. In our scoped harness on the in-page deletion category of the Nemetz standardized corpus, our recall is 0.833 to the best-performing competitor's 0.798 at precision 1.000, and across the evaluated corpus the carver reports no still-live row as deleted. Every figure is reproducible from the open implementation.

1 Introduction

SQLite ships inside every major browser, mobile operating system, and messaging application, making a SQLite file among the most common artifacts in a digital investigation [1]. The records a custodian *deleted*—a cleared chat, a removed history entry, a purged transaction—are frequently the records that matter, and they often still reside, intact or partially intact, in the file's free space, in an uncheckpointed write-ahead log, or in a rollback journal. The ordinary access path cannot reach them: `sqlite3` and the language bindings traverse the *live* b-tree and stop, by design.

Recovering that residue is a mature subfield. Nemetz *et al.* published a standardized corpus and showed that no single tool reliably handles all SQLite corner cases [2]; Pawlaszczyk and Hummert [3] and Meng and Baier [4] contributed carving tools and techniques; and the survey of Lee, Park, Lee and Choi [5] organizes the field into three families of technique—metadata-based, carving-based, and WAL-based recovery—and evaluates representative open-source tools on recovery rate, reliability, and false positives. Two gaps motivate this work. First, the literature treats the storage *conventions* that decide whether residue exists at all—WAL versus rollback journaling, `VACUUM`, and `secure_delete`—unevenly, even though they dominate real-case outcomes. Second, recovery tools differ sharply in whether they ever mistake a *live* row for a deleted one, and that false-positive behaviour is rarely controlled by design.

This paper addresses both. We give an account of the storage model and the conventions (§2–§3); we describe a set of recovery mechanisms (§4) constrained by a live-row exclusion invariant (§5); we present an open reference implementation (§6); and we evaluate it against four independent oracles on third-party corpora with published ground truth (§7–§8).

Contributions

1. An account of SQLite's on-disk deletion behaviour and the free-space lifetime conventions (journal vs. WAL, `VACUUM`, `secure_delete`), with an application case study of WeChat (§2–§3).
2. A live-row re-read exclusion invariant and an open, `forbid(unsafe)` reference implementation, our carver(§5–§6).
3. A reproducible multi-oracle evaluation—accuracy, false positives, and throughput—on independent corpora with published ground truth (§7–§8).

2 Where Deleted Data Lives

The SQLite database file is a sequence of fixed-size *pages* [6]. An ordinary rowid table is a b-tree whose leaf pages hold *cells*, one per row; each cell is a varint-prefixed record of serial-typed values keyed by an integer `rowid`. (We scope recovery and evaluation to ordinary rowid tables; `WITHOUT ROWID` tables store records in index b-trees and are out of scope here.) Recovery hinges on what happens to a cell's bytes once the row is no longer reachable from the b-tree.

Free blocks and the freelist

Normally—that is, with `secure_delete` off (§3)—a deleted row's bytes are not erased. The cell is unlinked from the page's cell-pointer array, and its span is either added to that page's intra-page *free-block* chain—a linked list of freed gaps inside an otherwise live page—or, when a page empties completely, the page is placed on the database *freelist* of reusable pages [6]. In both cases the original record bytes survive until that space is next *allocated* and overwritten. That window is what every carving technique exploits. A freed cell's first bytes (the leading payload-length and rowid varints) may be partly overwritten by the free-block header that now occupies the gap; the surviving tail still carries the record's serial-type array and values.

Overflow pages

A value too large for one page spills onto a chain of *overflow* pages linked by a four-byte next-page pointer [6]. A deleted row with a long `TEXT` or `BLOB` column therefore leaves residue spread across several pages—recoverable only by following, or reconstructing, the chain through the freelist.

The rollback journal and the write-ahead log

SQLite offers two crash-recovery mechanisms, and each leaves a distinct, recoverable side file. The *rollback journal* (`<db>-journal`) holds the *before*-images of pages a transaction is about to modify, so an aborted transaction can be rolled back. The *write-ahead log* (`<db>-wal`), available since SQLite 3.7.0 (2010) [7], inverts this: new page versions are *appended* to the WAL and only later *checkpointed* back into the main file. The two files are temporal opposites—the journal holds the pre-transaction state, the WAL the post-transaction states—and both can carry recoverable residue.

A `-wal` holds many commits. A WAL begins with a 32-byte header (a magic number, the page size, and two random *salt* values) followed by *frames*, each a 24-byte frame-header plus one page image. A frame whose header carries a non-zero "database size in pages" field is a *commit* marker; a single WAL "can and usually does record multiple transactions" [7]. Each frame copies the *current* WAL-header salts, and the salts change when the WAL is *reset* after a checkpoint (salt-1 incremented, salt-2 re-randomized), so frames left from a superseded WAL generation cannot be mistaken for current ones [6]. Checkpointing is automatic once the WAL reaches a threshold (1000 pages by default) [7]. Until then, the WAL retains the *commit history*: a page version a later commit superseded, or a row a later commit deleted, survives in the earlier frames.

3 Conventions That Govern Residue

Whether deleted data is recoverable is decided less by the deletion itself than by three conventions: which journaling mode the application uses, whether `VACUUM` has run, and whether `secure_delete` is enabled.

Rollback journaling vs. WAL

SQLite's *library* default remains rollback journaling in `DELETE` mode (the journal file is deleted at commit); the other rollback modes are `TRUNCATE`, `PERSIST` (the journal is kept but its header zeroed), `MEMORY`, and `OFF` [8]. Many modern applications, however, *opt into* WAL mode for concurrency, so in practice a `-wal` sidecar is the common case for mobile and desktop application databases while a persistent `-journal` is comparatively legacy. The distinction matters: a `DELETE`-mode journal is removed at commit and leaves nothing on a clean shutdown, whereas a `PERSIST` journal or an uncheckpointed WAL is a durable record of prior state, and the WAL holds *multiple* commits.

`VACUUM` is anti-forensic

`VACUUM` "rebuilds the database file, repacking it into a minimal amount of disk space" [9], discarding the freelist and free-block slack that carving relies on. The SQLite documentation is explicit: recoverable residue "can allow deleted content to be recovered … Running `VACUUM` will clean the database of all traces of deleted content" [9]. The related `auto_vacuum` setting reclaims emptied pages but does not repack partial pages: `auto_vacuum=FULL` truncates freed pages from the file at each commit, destroying whole-page freelist residue while leaving intra-page free-block residue; `auto_vacuum=INCREMENTAL` stores pointer-map pages but reclaims free pages only when `PRAGMA incremental_vacuum` is invoked [8].

`PRAGMA secure_delete`

With `secure_delete=ON`, SQLite overwrites the freed content of ordinary b-tree pages with zeros at deletion time; it is *off* by default, and a `FAST` variant zeroes content only when doing so adds no I/O, so it can still leave freelist traces [8]. The setting also does not govern the shadow tables of virtual tables

such as FTS, which can retain recoverable tokens even when `secure_delete` is enabled—a point that recurs in the WeChat case below. An investigator should therefore treat recoverable residue as evidence that `secure_delete` was *not* fully in force, and its absence as inconclusive rather than as proof of secure deletion.

Application case study: 微信 (WeChat)

WeChat illustrates how application conventions shape recovery. On Android, chat history is stored in an SQLCipher-encrypted database `EnMicroMsg.db`; on iOS the store is `MM.sqlite`, often unencrypted [10]. Hur *et al.* show that deleted WeChat messages can be recovered from the tokenized index of the full-text-search database (`FTS5IndexMicroMsg.db`) even after the message row is gone—precisely the FTS shadow-table residue that `secure_delete` does not cover—and give a decryption method for the IMEI-absent case [10]. Once such a database is decrypted to plaintext SQLite, the deleted rows live in the same free blocks, overflow chains, and WAL frames as any other SQLite artifact, and the mechanisms below apply.

4 Recovery Mechanisms

Following the technique families of the Choi survey [5] (metadata-based, carving-based, WAL-based) and adding rollback-journal recovery, we implement the mechanisms below. Each emits a labelled *recovery source* and a confidence (§5); each respects the live-row exclusion invariant.

Free-block and freelist carving (metadata- and carving-based). Whole pages on the freelist contain only deallocated content and are the strongest case; intra-page free blocks are gaps in still-live pages and are more likely to be partly overwritten. We carve both, decoding each candidate cell against the live schema and reading every value in its native serial type.

Exact-tile free-block reconstruction. When a freed cell's leading bytes are destroyed by the free-block header, the cell length and `rowid` are gone but the serial-type tail survives. We rebuild the record from the surviving tail plus the owning table's schema template, accepting a reconstruction only when it *exactly tiles* the free block—its decoded extent must end on the block boundary. This exactness test rejects column-shifted candidates, and it extends to two-byte `rowid`s (≥ 128) and to runs of coalesced freed cells, where each cell in the span must tile in turn. The destroyed `rowid` is reported as absent rather than guessed.

Overflow-chain reassembly (carving-based). A deleted row whose payload spilled onto overflow pages is reassembled by following the freed chain through the freelist leaves to a byte-exact body. Because a freelist leaf can be stale, this path is graded below an intact in-page cell.

Dropped-table schema recovery (metadata-based). A dropped table's definition survives as a deleted `sqlite_master` row in page-1 free space. We recover the dropped table, index, view, or trigger name and its `CREATE` statement, which both documents the deletion and supplies a schema template for attributing other residue.

WAL commit-history overlay (WAL-based). We parse the WAL frame by frame, reconstruct the database state at each `COMMIT` marker, and surface rows that were live at an earlier commit but deleted by a later one. Each recovered version carries its log-sequence provenance (the two salts and the frame index), giving an ordered edit history. There is no wall-clock timestamp in a WAL—only this commit order.

Rollback-journal before-image folding (journal-based). When a `<db>-journal` is present we fold in the *before*-images of the last transaction: rows it deleted and the pre-edit values of rows it modified. This is the temporal inverse of the WAL overlay—where the WAL holds after-images, the journal holds the before-image of the last transaction.

Tier-2 fragment salvage. Where a full row cannot be reconstructed but a distinctive cell survives at a structural anchor (a `TEXT` value of at least four UTF-8 bytes, or a `REAL`), we salvage the maximal decodable column prefix as a *fragment*—a typed object with no `rowid` and an incomplete column set, kept in a separate output surface so it is not mistaken for a full recovered row.

5 The Live-Row Exclusion Invariant

In a forensic setting, reporting a still-*live* row as "deleted" is the costliest error: it fabricates evidence. We constrain the carver so that this specific outcome cannot occur, in two layers. The invariant concerns live-row re-reads only; it is not a claim of perfect precision—phantoms (coincidental byte sequences that decode to a plausible but spurious record) and schema misattribution remain possible and are measured, not assumed away (§8).

Structural exclusion. We compute the byte extents of every *live* cell directly from the b-tree structure. The set of carvable free regions is the *complement* of those extents within each page, so free-space carving never scans a region a live cell occupies.

Table 1: Confidence assigned by recovery path.

Conf.	Recovery path
0.9	full row, distinctive text (freelist / WAL frame / live-anchor)
0.72	in-page free block, distinctive text (0.9×0.8)
0.675	overflow-chain reassembly (0.9×0.75)
0.6	full row, no distinctive text
0.48	in-page free block, no distinctive text (0.6×0.8)
0.4	exact-tile free-block reconstruction
0.3	free-block reconstruction spilling to overflow (0.4×0.75)
0.2	Tier-2 fragment

Value-collision filter. The WAL, overflow, and journal paths recover from allocated pages rather than free regions, and a freed cell's bytes can in principle coincide with a live row's values. After recovery, any record whose decoded values equal those of a live row in the same table is dropped. What remains—for example an `UPDATE`'s freed *prior* version, whose values differ from the live row—is genuine deleted edit history.

Together these exclude a live-row re-read independently of any confidence threshold. Confidence (Table 1) is reported to rank residue, not to enforce the invariant. A base of 0.9 for a full row with distinctive text is attenuated by an in-page factor (0.8) for free-block residue and by an overflow-chain factor (0.75) for chain-reassembled bodies; reconstructions take a fixed 0.4 and fragments 0.2. The CLI exposes a `--min-confidence` ladder (`info` 0.0, `low` 0.2, `medium` 0.4, `high` 0.6, `critical` 0.8) that filters every emitted item; it is a recall/noise dial, not a safety control.

6 Reference Implementation

Our carver [11] is a Rust workspace of three crates: a panic-free, read-only file-format reader (b-tree, WAL, freelist, overflow); a carving and audit harness with the recovery oracles; and a command-line tool. All three are `forbid(unsafe)`; the project pins a current Rust toolchain, declares a conservative minimum supported version, and gates on 100 % function coverage in continuous integration. The reader opens evidence *read-only* and, for WAL artifacts, in immutable mode, so analysis does not mutate or checkpoint the file under examination.

Output is designed for review rather than reparsing. The default carve writes a per-rowid version-history workbook—live rows interleaved with the prior and deleted versions recovered from the WAL, the rollback journal, and free space, ordered by commit sequence—with image BLOBs rendered as in-cell thumbnails. A `-f db` option emits a queryable SQLite database whose recovered rows are attributed in three tiers (observed fact, forensic inference, unknown), and `-f jsonl/csv` stream the raw records. Every record carries provenance columns: page, byte offset, `rowid` (or absent), recovery source, and confidence.

7 Validation Corpus

A recovery tool validated only against fixtures its own authors built inherits their blind spots. We therefore validate primarily against third-party corpora whose *ground truth was authored by someone else*, and label every result by the strength of the check that vouches for it (Table 2). Each corpus entry, with source URL, hash, and answer-key provenance, is recorded in the implementation's corpus catalogue [11].

The head-to-head accuracy results below use the Nemetz standardized corpus [2], whose authors published a per-row answer key for every deleted record. We score the three deletion categories with a format-stable identity (in-page deletion `0C`, deleted-then-overwritten `0D`, and deleted overflow `0E`); categories whose only key column is a floating-point value rendered differently by each tool are excluded from the cross-tool key, as are the whole-table-loss and anti-forensic categories that have no deleted-row anchor.

8 Evaluation

Accuracy: a multi-oracle head-to-head

We compare our carveragainst four independent recovery tools—`undark` (C) [14], `fqlite` (Java) [3], `bring2lite` (Python) [4], and DC3's `sqlite_dissect` (Python) [13]—each scored against the *same* Nemetz answer key with the same two-column identity matcher (Table 4). Substrate recall (R_{sub}) is computed on the rows whose identity still survives in the artifact; end-to-end recall (R_{e2e}) is against all originally-deleted rows, including those whose identity a later write destroyed; precision counts phantom rows. We drive `fqlite` through a headless source-instrumentation tap of its carving engine and the two Python tools through normalizing wrappers; every tool is pinned to an exact release or commit and every patch, shim, and the tap is committed, so the comparison is reproducible from a clean checkout (Table 3) [11].

Exact versions and the `fqlite` instrumentation. Table 3 pins each tool to a release or commit, states how it was driven, and names the committed artifact a reader re-runs it from; the fixtures themselves are built with the stock `sqlite3` shell (3.45.3). The other tools run essentially as published—`undark`'s patch is two behavior-preserving build-portability hunks (clang/macOS), and `bring2lite` needs only a headless Qt import shim and the replacement of deprecated `is`-literal comparisons; neither alters

Table 2: Evidence tiers. Tier 1 = an independent party authored both the artifact and the answer key (or it is real-device data); Tier 2 = real `sqlite3`-engine bytes whose ground truth is derivable from the documented construction or confirmed by an independent oracle. Per-asset provenance (URLs, hashes) is in the corpus catalogue [11].

Tier	Corpus / oracle	What it establishes
1	Nemetz standardized corpus [2] (CC0, per-row `.xml` keys)	deleted-record recall & precision vs. third-party ground truth
1	NIST CFReDS / CFTT SQLite sets [12] (SFT-01/03/05)	text encodings; WAL & journal recovery; native types + BLOBs
1	NIST CFReDS *Data Leakage* (`snapshot.db` from a Volume Shadow Copy) [12]	real-case end-to-end recovery of two deleted rows, one freeblock-clobbered
1	DC3 `sqlite_dissect` corpus [13] (no-deletion baseline)	no false positives on unallocated content
1	`sqlite-unhide` corpus (independent `.txt` keys) [11]	two-byte-rowid free-block recovery; surfaced two real defects
1	genuine iOS-17 application databases [11]	no-panic robustness over real device data
2	real-`sqlite3` fixtures; `sqlite3 .recover`; `calamine` XLSX reader	construction-derivable answers; differential agreement; output round-trip

Table 3: Exact versions of the compared tools and how each was driven. Every patch, the headless `fqlite` tap and its stubs, the Python wrappers/shims, and per-tool setup recipes are in the artifact for independent verification [11]; the fixtures are built with `sqlite3` 3.45.3.

Tool	Version / commit	Driven via
`undark`	0.7.1	native binary
`fqlite`	4.22 (`26922bd`)	headless source tap
`bring2lite`	commit `e876bf2`	wrapper + PyQt5 shim
`sqlite_dissect`	1.0.0 (DC3)	wrapper (`-c -f`)

Table 4: Deleted-record recovery on the Nemetz corpus (categories `0C`/`0D`/`0E`). R_{sub} = recall on the surviving-identity substrate; R_{e2e} = recall on all originally-deleted rows. Recall/TP are harness-computed against the published answer key; competitor precision is measured through our committed wrappers/tap.

Cat.	Tool	TP	FP	FN	R_{sub}	R_{e2e}	Prec.
`0C`	**ours**	**70**	**0**	14	**0.833**	**0.833**	**1.000**
	fqlite	67	17	17	0.798	0.798	0.798
	bring2lite	23	33	61	0.607	0.607	0.689
	sqlite_dissect	40	0	44	0.476	0.476	1.000
	undark	14	10	70	0.167	0.167	0.583
`0D`	**ours**	19	0	0	**1.000**	0.422	**1.000**
	fqlite	20†	0	0	1.000	0.444	1.000
	sqlite_dissect	18	3	2	0.895	0.400	0.857
	bring2lite	7	0	12	0.368	0.156	1.000
	undark	1	10	18	0.053	0.022	0.091
`0E`	**ours**	**4**	**0**	0	**1.000**	**0.333**	**1.000**
	undark	3	6	1	0.750	0.250	0.333
	bring2lite	3	5	1	0.750	0.250	0.375
	sqlite_dissect	3	633	1	0.750	0.250	0.005
	fqlite	2	6	2	0.500	0.167	0.250

† `fqlite`'s two-column projection matches 20 distinct identities on the 19-row `0D` substrate; substrate recall is capped at 1.000.

recovery logic—and `sqlite_dissect` is the stock DC3 release run with carving enabled (`-c -f`). `fqlite` alone required engineering, because its command-line mode was removed at v2.0: we drive the carving engine directly through a committed, GUI-free tap. `HeadlessTap` constructs an `fqlite.base.Job`, runs `Job.run(path)` synchronously, and reads the engine's `resultlist`—the same structure the GUI presents—emitting the recovered deleted rows as CSV without ever opening a JavaFX window. It requires *no edits to fqlite's own source*: at the pinned commit (`26922bd`) every `gui.add_table(...)` call is already behind an `if (gui != null)` guard upstream, so the GUI-null engine path runs to completion; the only build-time additions are six compile-only stubs standing in for fqlite's optional LLM-assistant packages so the engine links without langchain4j/llama.cpp. The tap builds with OpenJDK 25 (`--release 21`) against OpenJFX 22.0.2. The driver (`tools/fqlite/tap/HeadlessTap.java`), its run wrapper, the stubs, and a full reproduction recipe (`tools/fqlite/SETUP.md`) are committed for independent verification [11]. `fqlite`'s WAL reader, by contrast, is instantiated only on a GUI code path, which is why category `10`'s WAL case is cited from the original paper rather than re-measured through the tap.

In this harness, on in-page deletion (`0C`) our carver recovers 70 of 84 surviving-identity rows (R_{sub} = 0.833) to `fqlite`'s 67 (0.798), at precision 1.000—no phantom rows and no live re-reads—against `fqlite`'s 0.798. On deleted-then-overwritten rows (`0D`) our carver and `fqlite` both recover every surviving-identity row at precision 1.000; on the small overflow substrate (`0E`) our carver recovers all four at precision 1.000, including one reassembled overflow chain. End-to-end recall is lower (`0D` 0.422, `0E` 0.333) because many originally-deleted rows had their identity overwritten—an effect the substrate/e2e split makes explicit. Across the evaluated categories our carver reports no still-live row as deleted, whereas

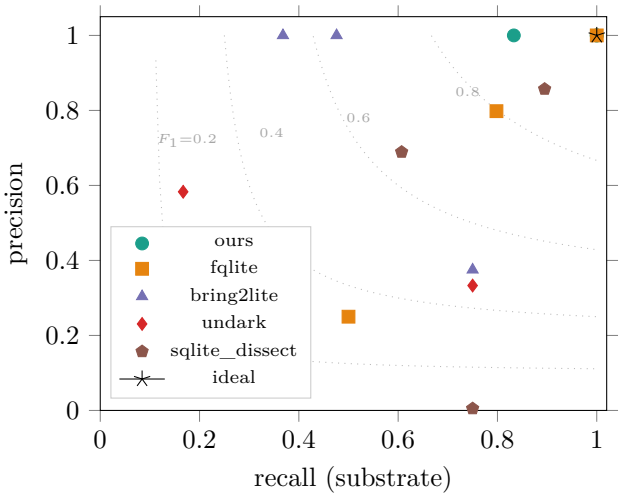


Figure 1: Substrate precision–recall across categories `0C`/`0D`/`0E` with constant-F_1 contours. In this harness our carver lies on precision 1.000 in each category.

`undark`, which dumps live and deleted rows together, reports 56 live rows as deleted on `0D` alone, and `sqlite_dissect`, with carving enabled, emits 633 phantoms and re-reads 7 live rows on `0E` (precision 0.005)—the false-positive hazard the exclusion invariant removes. Figure 1 plots substrate precision and recall against constant-F_1 contours; Figure 2 breaks out the F_1 and $F_{0.5}$ scores by category.

False positives across forensic hazards

Accuracy on a clean corpus says little about the hazard that matters most—reporting live data as deleted. We replicate the three adversarial scenarios of the Choi survey [5] on real `sqlite3`-engine bytes (a Tier-2 *replication* of the published construction, not the authors' own files): `0F`, where a b-tree rebalance moves a live row onto a freed page; `0B`, a drop-and-recreate of a same-schema table; and `10`, a `secure_delete` wipe of the main image leaving residue only in the WAL. On `0F`, our carverrecovers 33 of 50 with no live false positives, where `bring2lite` reports 13 live rows as deleted; on `10`, the WAL-only scenario, both WAL-aware tools recover all 20, while the main-image-only tools recover none.

Throughput

On a generated 100 MB database with 80,000 deleted rows (single machine, median of five runs), our carverstreaming JSONL recovers 79,904 deleted rows ($R_{\text{e2e}} = 0.9988$) with no live false positives in 15.6 s. The faster tools do different work: `undark` dumps 177,906 rows—live and deleted together, undifferentiated—in 1.5 s. Throughput is not directly comparable across tools that recover different things; we report it for completeness.

9 Related Work

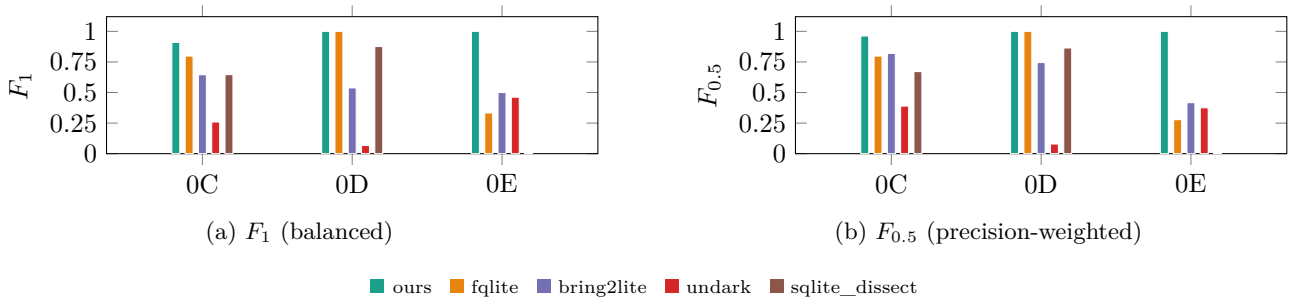
The Choi survey [5] is the closest prior work: it organizes SQLite deleted-record recovery into metadata-based, carving-based, and WAL-based families and evaluates the same open-source tools we compare against, with attention to false positives. We add a live-row exclusion invariant and a reproducible head-to-head on the standardized corpus. Nemetz *et al.* [2] contributed the standardized corpus and the finding that no tool handles every corner case—which our multi-oracle evaluation re-confirms tool by tool. `fqlite` [3] and `bring2lite` [4] recover free-block and freelist residue; `undark` [14] dumps all intact records without separating live from deleted; and DC3's `sqlite_dissect` [13] parses the database together with its journal and WAL. On the application side, Hur *et al.* [10] recover deleted WeChat messages from full-text-search index residue, complementary to the free-space and WAL recovery treated here.

10 Discussion and Limitations

Recovery is bounded by the conventions of §3: `VACUUM` and `secure_delete` destroy residue, and an allocation that overwrites a freed cell ends its recoverability. End-to-end recall on overflow bodies is 0.333 on `0E` because most deleted overflow payloads in the corpus were overwritten; the substrate recall of 1.000 separates what survived from what did not. The evaluation is scoped to selected Nemetz categories and to surviving-identity substrates, and to ordinary rowid tables; competitor precision is measured through our own wrappers and tap, so a tool's phantom count reflects its emission policy under our harness, which we re-derived for `fqlite`. The false-positive scenarios are a replication of the Choi survey's construction on real-engine bytes rather than the authors' own files. The exclusion invariant covers live-row re-reads, not phantom suppression or schema-attribution correctness, which the measured precision reports.

11 Conclusion

Deleted SQLite records persist in free blocks, overflow chains, rollback journals, and the multiple uncheckpointed commits of a write-ahead log, until `VACUUM`, `secure_delete`, or reallocation erases them. We described a set of recovery mechanisms for that residue, constrained so a still-live row is not reported as deleted, and an open `forbid(unsafe)` reference implementation. Measured against four independent tools on third-party corpora with published ground truth, the implementation leads on in-page substrate recall at precision 1.000 in our harness and reports no live row as deleted across the evaluated categories. The implementation, corpora provenance, and the exact harness commands are open for reuse and re-measurement.



(a) F_1 (balanced)

(b) $F_{0.5}$ (precision-weighted)

Figure 2: F_1 and $F_{0.5}$ by category and tool, on substrate recall. $F_{0.5}$ weights precision over recall—the forensic preference, since a phantom row costs an examiner more than a missed low-confidence one. Bars are computed from the same harness figures as Table 4.

References

[1] SQLite Consortium. Most widely deployed and used database engine. https://www.sqlite.org/mostdeployed.html, 2026. Accessed 2026-06.

[2] Sebastian Nemetz, Sven Schmitt, and Felix C. Freiling. A standardized corpus for SQLite database forensics. *Digital Investigation*, 24:S121–S130, 2018. doi: 10.1016/j.diin.2018.01.015. Proceedings of DFRWS EU 2018.

[3] Dirk Pawlaszczyk and Christian Hummert. Making the invisible visible—techniques for recovering deleted SQLite data records. *International Journal of Cyber Forensics and Advanced Threat Investigations*, 1(1–3):27–41, 2021. doi: 10.46386/ijcfati.v1i1-3.17.

[4] Christian Meng and Harald Baier. bring2lite: A structural concept and tool for forensic data analysis and recovery of deleted SQLite records. *Digital Investigation*, 29:S31–S41, 2019. doi: 10.1016/j.diin.2019.04.017. Proceedings of DFRWS USA 2019.

[5] Seonghyeon Lee, Sooyoung Park, Insoo Lee, and Jongmoo Choi. A comprehensive analysis and evaluation of SQLite deleted record recovery techniques: A survey. *Forensic Science International: Digital Investigation*, 55:302031, 2025. doi: 10.1016/j.fsidi.2025.302031.

[6] SQLite Consortium. The SQLite database file format. https://www.sqlite.org/fileformat2.html, 2026. Accessed 2026-06.

[7] SQLite Consortium. Write-ahead logging. https://www.sqlite.org/wal.html, 2026. Accessed 2026-06; WAL available since 3.7.0 (2010-07-21).

[8] SQLite Consortium. Pragma statements supported by SQLite. https://www.sqlite.org/pragma.html, 2026. Accessed 2026-06; see `secure_delete`, `auto_vacuum`, `journal_mode`.

[9] SQLite Consortium. VACUUM. https://www.sqlite.org/lang_vacuum.html, 2026. Accessed 2026-06.

[10] Uk Hur, Myungseo Park, and Jongsung Kim. Study on improved decryption method of WeChat messenger and deleted message recovery using SQLite full text search data. *Journal of the Korea Institute of Information Security and Cryptology*, 30(3):405–415, 2020. doi: 10.13089/JKIISC.2020.30.3.405.

[11] Anonymous Author(s). Reference implementation (anonymized for double-blind review). https://anonymous.4open.science/r/ANON-XXXX/, 2026. Anonymized artifact for review; the corpus catalogue, harness, and committed competitor setup are in the anonymized repository.

[12] National Institute of Standards and Technology. Computer forensic reference data sets (cfreds): SQLite data recovery. https://cfreds.nist.gov/, 2021. CFTT SQLite recovery read-me https://www.nist.gov/document/cftt-document-sqlite-recovery-read-me; accessed 2026-06.

[13] DoD Cyber Crime Center (DC3). SQLite dissect. https://github.com/dod-cyber-crime-center/sqlite-dissect, 2026. Accessed 2026-06.

[14] Paul L. Daniels. undark: an SQLite deleted and corrupted data-recovery tool. https://pldaniels.com/undark/, 2026. Code mirror https://github.com/inflex/undark; accessed 2026-06.